Verapay Technical Docs

Backend Engineering Documentation

SERVER ARCHITECTURE, DATABASE SCHEMA & API REFERENCE • VERSION 2.0

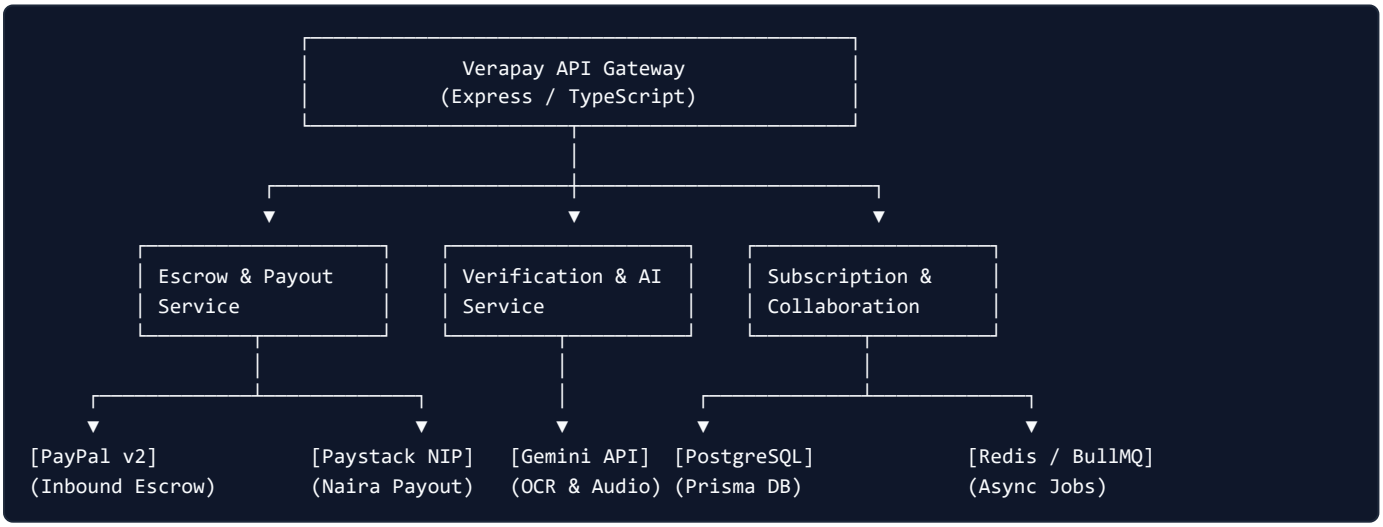
Verapay Platform
Verify, then release.

Verapay: Backend Engineering Documentation

Server Architecture, Database Schema, API Specifications & AI Pipeline | Version 2.0

1. Backend Technology Stack & Architecture

Component	Technology	Role in System
Runtime / API	**Node.js (TypeScript) / Express** or **FastAPI**	REST API gateway, authentication, and core business orchestration.
Primary Database	**PostgreSQL 16**	ACID-compliant relational storage for financial ledgers, escrows, and audits.
ORM	**Prisma ORM**	Type-safe database queries, schema migrations, and relational modeling.
Job Queue & Cache	**Redis 7 + BullMQ**	Asynchronous worker queues for AI OCR jobs, quote comparisons, and webhooks.
AI Intelligence	**Google Gemini 1.5/2.0 Flash API**	Multimodal document OCR, voice note transcription, and price reasoning.
Media Forensics	**Sharp + dHash (Perceptual Hashing)**	Image metadata analysis, EXIF extraction, and duplicate photo detection.
Payment Inbound	**PayPal REST API (v2)**	Checkout Orders API, Escrow Authorization / Capture, Refund API.
Payment Outbound	**Paystack / Flutterwave API**	NIP NUBAN Account Resolution, Transfers API, live Naira bank settlements.



2. PostgreSQL Database Schema (Prisma Schema)

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum UserRole {
  DIASPORA_SENDER
  FAMILY_MEMBER
  CONTRACTOR
  FIELD_SURVEYOR
  ADMIN_OPERATOR
}

enum PlanType {
  BASIC
  VERAPAY_PLUS
  VERAPAY_PROJECTS
}

enum EscrowStatus {
  PENDING_FUNDING
  ESCROW_HELD
  VERIFYING
  READY_FOR_APPROVAL
  PAYOUT_INITIATED
  COMPLETED
  REFUNDED
  DISPUTED
}

enum RiskSignal {
  GREEN
  AMBER
  RED
}

enum PayeeCategory {
  SCHOOL
  HOSPITAL
  LANDLORD
  BUILDING_SUPPLIER
  ARTISAN_INFORMAL
}

model User {
  id               String            @id @default(uuid())
  email            String?           @unique
  phoneNumber      String            @unique
  fullName         String
  role             UserRole          @default(DIASPORA_SENDER)
  countryCode      String            // "GB", "US", "CA", "NG"
  activePlan       PlanType          @default(BASIC)
  createdAt        DateTime          @default(now())
  updatedAt        DateTime          @updatedAt

  paymentsSent     Payment[]         @relation("SenderPayments")
  familyMemberships FamilyMember[]
  createdProjects  Project[]         @relation("SenderProjects")
  potContributions PotContribution[]
}
```

```
model FamilyGroup {
  id                String            @id @default(uuid())
  name              String            // e.g., "The Balogun Family"
  creatorId         String
  createdAt         DateTime          @default(now())

  members           FamilyMember[]
  sharedPots        SharedPot[]
}

model FamilyMember {
  id                String            @id @default(uuid())
  familyGroupId     String
  userId            String
  isBeneficiary     Boolean           @default(false)
  joinedAt          DateTime          @default(now())

  familyGroup       FamilyGroup       @relation(fields: [familyGroupId], references: [id])
  user              User              @relation(fields: [userId], references: [id])
}

model Payee {
  id                String            @id @default(uuid())
  businessName      String
  category          PayeeCategory
  isCacVerified     Boolean           @default(false)
  cacRegistrationNo String?
  bankCode          String            // e.g., "058" for GTBank
  bankName          String
  accountNumber     String            // 10-digit NUBAN
  accountName       String            // Verified NIP Name
  address           String?
  phoneNumber       String?
  isPreVerified     Boolean           @default(false)
  createdAt         DateTime          @default(now())

  payments          Payment[]
}

model Payment {
  id                String            @id @default(uuid())
  senderId          String
  payeeId           String
  milestoneId       String?
  amountUSD         Decimal           @db.Decimal(12, 2)
  amountNGN         Decimal           @db.Decimal(14, 2)
  exchangeRate      Decimal           @db.Decimal(10, 4)
  feeUSD            Decimal           @db.Decimal(8, 2)
  escrowStatus      EscrowStatus      @default(PENDING_FUNDING)
  paypalOrderId     String?           @unique
  paypalCaptureId   String?           @unique
  paystackRef       String?           @unique
  idempotencyKey    String            @unique

  createdAt         DateTime          @default(now())
  updatedAt         DateTime          @updatedAt

  sender            User              @relation("SenderPayments", fields: [senderId], references: [id])
  payee             Payee             @relation(fields: [payeeId], references: [id])
  verificationJob   VerificationJob?
  evidenceMedia     EvidenceMedia[]
}

model VerificationJob {
  id                String            @id @default(uuid())
  paymentId         String            @unique
```

```
  riskSignal        RiskSignal         @default(AMBER)
  payeeCheckPassed  Boolean            @default(false)
  billCheckPassed   Boolean            @default(false)
  proofCheckPassed  Boolean            @default(false)
  aiSummaryReport   String             @db.Text
  duplicateHashHits Int                @default(0)
  priceDeltaPercent Decimal?           @db.Decimal(5, 2)
  requiresHumanHelp Boolean            @default(false)
  createdAt         DateTime           @default(now())
  completedAt       DateTime?

  payment           Payment            @relation(fields: [paymentId], references: [id])
}

model EvidenceMedia {
  id                String             @id @default(uuid())
  paymentId         String
  mediaType         String             // "INVOICE", "LIVE_PHOTO", "SURPRISE_VIDEO", "RECEIPT"
  fileUrl           String
  perceptualHash    String?            // dHash string for image replay catching
  latitude          Decimal?           @db.Decimal(10, 7)
  longitude         Decimal?           @db.Decimal(10, 7)
  capturedAt        DateTime?
  uploadedAt        DateTime           @default(now())

  payment           Payment            @relation(fields: [paymentId], references: [id])
}

model Project {
  id                String             @id @default(uuid())
  ownerId           String
  title             String             // e.g., "4-Bedroom Duplex in Ibadan"
  locationCity      String
  totalBudgetUSD    Decimal            @db.Decimal(12, 2)
  createdAt         DateTime           @default(now())

  owner             User               @relation("SenderProjects", fields: [ownerId], references: [id])
  milestones        Milestone[]
}

model Milestone {
  id                String             @id @default(uuid())
  projectId         String
  sequenceNumber    Int
  title             String             // "Foundation & German Floor"
  targetBudgetUSD   Decimal            @db.Decimal(12, 2)
  status            EscrowStatus       @default(PENDING_FUNDING)
  createdAt         DateTime           @default(now())

  project           Project            @relation(fields: [projectId], references: [id])
}

model SharedPot {
  id                String             @id @default(uuid())
  familyGroupId     String
  title             String             // "Dad's Eye Surgery at LUTH"
  targetAmountUSD   Decimal            @db.Decimal(12, 2)
  currentAmountUSD  Decimal            @default(0) @db.Decimal(12, 2)
  status            String             // "COLLECTING", "TARGET_REACHED", "DISBURSED"
  createdAt         DateTime           @default(now())

  familyGroup       FamilyGroup        @relation(fields: [familyGroupId], references: [id])
  contributions     PotContribution[]
}

model PotContribution {
  id                String             @id @default(uuid())
```

```
  potId             String
  userId            String
  amountUSD         Decimal          @db.Decimal(10, 2)
  paypalCaptureId   String           @unique
  contributedAt     DateTime         @default(now())

  pot               SharedPot        @relation(fields: [potId], references: [id])
  user              User             @relation(fields: [userId], references: [id])
}
```

3. Core Verification & AI Service Pipeline

3.1. Multimodal Document OCR Engine (Gemini Flash)

```
// src/services/ai/InvoiceOcrService.ts
import { GoogleGenAI } from '@google/genai';

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

export interface ExtractedInvoice {
  institutionName: string;
  studentOrPatientName?: string;
  invoiceNumber?: string;
  invoiceDate?: string;
  totalAmountNGN: number;
  lineItems: Array<{
    description: string;
    quantity: number;
    unitPrice: number;
    total: number;
  }>;
  bankDetails?: {
    accountNumber: string;
    bankName: string;
    accountName: string;
  };
}

export async function parseInvoiceWithGemini(fileBuffer: Buffer, mimeType: string): Promise<ExtractedInvoice> {
  const prompt = `
    Analyze this Nigerian institutional invoice / receipt.
    Extract the following structured JSON:
    {
      "institutionName": "String",
      "studentOrPatientName": "String or null",
      "invoiceNumber": "String or null",
      "invoiceDate": "YYYY-MM-DD or null",
      "totalAmountNGN": Number,
      "lineItems": [{"description": "String", "quantity": Number, "unitPrice": Number, "total": Number}],
      "bankDetails": {"accountNumber": "String", "bankName": "String", "accountName": "String"}
    }
    Ensure all numbers are clean floats. Verify that line item sums equal the total.
  `;

  const response = await ai.models.generateContent({
    model: 'gemini-1.5-flash',
    contents: [
      {
        role: 'user',
        parts: [
          { text: prompt },
          { inlineData: { data: fileBuffer.toString('base64'), mimeType } }
        ]
      }
    ],
    config: { responseMimeType: 'application/json' }
  });

  return JSON.parse(response.text) as ExtractedInvoice;
}
```

3.2. Media Forensics & Perceptual Hashing (dHash)

```
// src/services/forensics/PerceptualHashService.ts
import sharp from 'sharp';

export async function computeImageDHash(imageBuffer: Buffer): Promise<string> {
  // Resize to 9x8 grayscale
  const rawPixels = await sharp(imageBuffer)
    .resize(9, 8, { fit: 'fill' })
    .grayscale()
    .raw()
    .toBuffer();

  let hash = '';
  for (let row = 0; row < 8; row++) {
    for (let col = 0; col < 8; col++) {
      const leftPixel = rawPixels[row * 9 + col];
      const rightPixel = rawPixels[row * 9 + col + 1];
      hash += leftPixel < rightPixel ? '1' : '0';
    }
  }

  // Convert 64-bit binary to 16-character hex
  let hexHash = '';
  for (let i = 0; i < 64; i += 4) {
    hexHash += parseInt(hash.substring(i, i + 4), 2).toString(16);
  }
  return hexHash;
}

export function computeHammingDistance(hash1: string, hash2: string): number {
  let distance = 0;
  for (let i = 0; i < hash1.length; i++) {
    const val = parseInt(hash1[i], 16) ^ parseInt(hash2[i], 16);
    distance += val.toString(2).split('1').length - 1;
  }
  return distance; // Distance < 5 indicates an identical or re-uploaded photo
}
```

3.3. Paystack NUBAN & CAC Validation Service

```
// src/services/payee/NubanVerificationService.ts
import axios from 'axios';

export async function verifyNubanAccount(accountNumber: string, bankCode: string) {
  const url = `https://api.paystack.co/bank/resolve?account_number=${accountNumber}&bank_code=${bankCode}`;
  const response = await axios.get(url, {
    headers: { Authorization: `Bearer ${process.env.PAYSTACK_SECRET_KEY}` },
  });

  if (!response.data.status) {
    throw new Error('NUBAN Account verification failed.');
  }

  return {
    accountNumber: response.data.data.account_number,
    accountName: response.data.data.account_name, // e.g. "BRIGHT FUTURE ACADEMY LTD"
  };
}
```

4. REST API Specification

4.1. Payments & Escrow Endpoints

`POST /api/v1/payments/create`

- **Description:** Diaspora sender creates a new escrow payment intent.
- **Headers:** `Authorization: Bearer <token>`, `Idempotency-Key: <uuid>`
- **Request Body:**

```
{
  "payeeCategory": "SCHOOL",
  "payeeName": "Bright Future School",
  "bankCode": "058",
  "accountNumber": "0123456789",
  "amountUSD": 400.00,
  "invoiceDescription": "Ada Term 1 Tuition"
}
```

- **Response (201 Created):**

```
{
  "paymentId": "pay_9823f4a2",
  "escrowStatus": "PENDING_FUNDING",
  "amountUSD": 400.00,
  "feeUSD": 6.00,
  "totalChargedUSD": 406.00,
  "exchangeRate": 1650.00,
  "amountNGN": 660000.00,
  "paypalClientOrderId": "ORDER_7721998A"
}
```

`POST /api/v1/payments/:id/approve`

- **Description:** Sender approves payout following Gate 4 Green/Amber inspection.
- **Headers:** `Authorization: Bearer <token>`
- **Response (200 OK):**

```
{
  "paymentId": "pay_9823f4a2",
  "escrowStatus": "PAYOUT_INITIATED",
  "paystackTransferCode": "TRF_99182371a",
  "message": "Naira payout has been dispatched to Bright Future School NUBAN."
}
```

`POST /api/v1/payments/:id/refund`

- **Description:** Sender rejects verification or proof failed (1-click refund).
- **Headers:** `Authorization: Bearer <token>`
- **Response (200 OK):**

```
{
  "paymentId": "pay_9823f4a2",
  "escrowStatus": "REFUNDED",
  "refundId": "REF_77192388A",
  "message": "100% of escrowed principal refunded to your PayPal account."
}
```

4.2. Verification & Forensics Endpoints

`POST /api/v1/verification/analyze-invoice`

- **Description:** Uploads an invoice or receipt for automated Gate 2 multimodal OCR and anti-duplicate hashing.

- **Form Data:** `file: <binary>`, `paymentId: String`
- **Response (200 OK):**

```
{
  "invoiceNumber": "INV-2026-091",
  "institutionName": "Bright Future School",
  "studentName": "Adaeze Okonkwo",
  "totalExtractedNGN": 660000.00,
  "isDuplicate": false,
  "riskRating": "GREEN"
}
```

`POST /api/v1/media/upload-live-photo`

- **Description:** Contractor uploads in-app live photo with watermarked telemetry.
- **Form Data:** `image: <binary>`, `milestoneId: String`, `lat: Float`, `lng: Float`, `timestamp: ISO`
- **Response (200 OK):**

```
{
  "mediaId": "med_8812390a",
  "perceptualHash": "a1f8902be34199c2",
  "isDuplicate": false,
  "locationVerified": true
}
```

5. Webhook Handlers & Concurrency Control

5.1. PayPal Webhook Listener (`POST /api/v1/webhooks/paypal`)

- Listens for `PAYMENT.CAPTURE.COMPLETED`.
- Validates webhook signature using PayPal SDK cert.
- Atomically updates `Payment.escrowStatus` to `ESCROW_HELD`.
- Enqueues `VerificationWorker` job in BullMQ queue.

5.2. Paystack Webhook Listener (`POST /api/v1/webhooks/paystack`)

- Listens for `transfer.success` and `transfer.failed`.
- Verifies `x-paystack-signature` HMAC-SHA512.
- On success, marks `Payment.escrowStatus = COMPLETED` and emails audited receipt to sender.
- On failure, rolls back to `ESCROW_HELD` and alerts human operations.